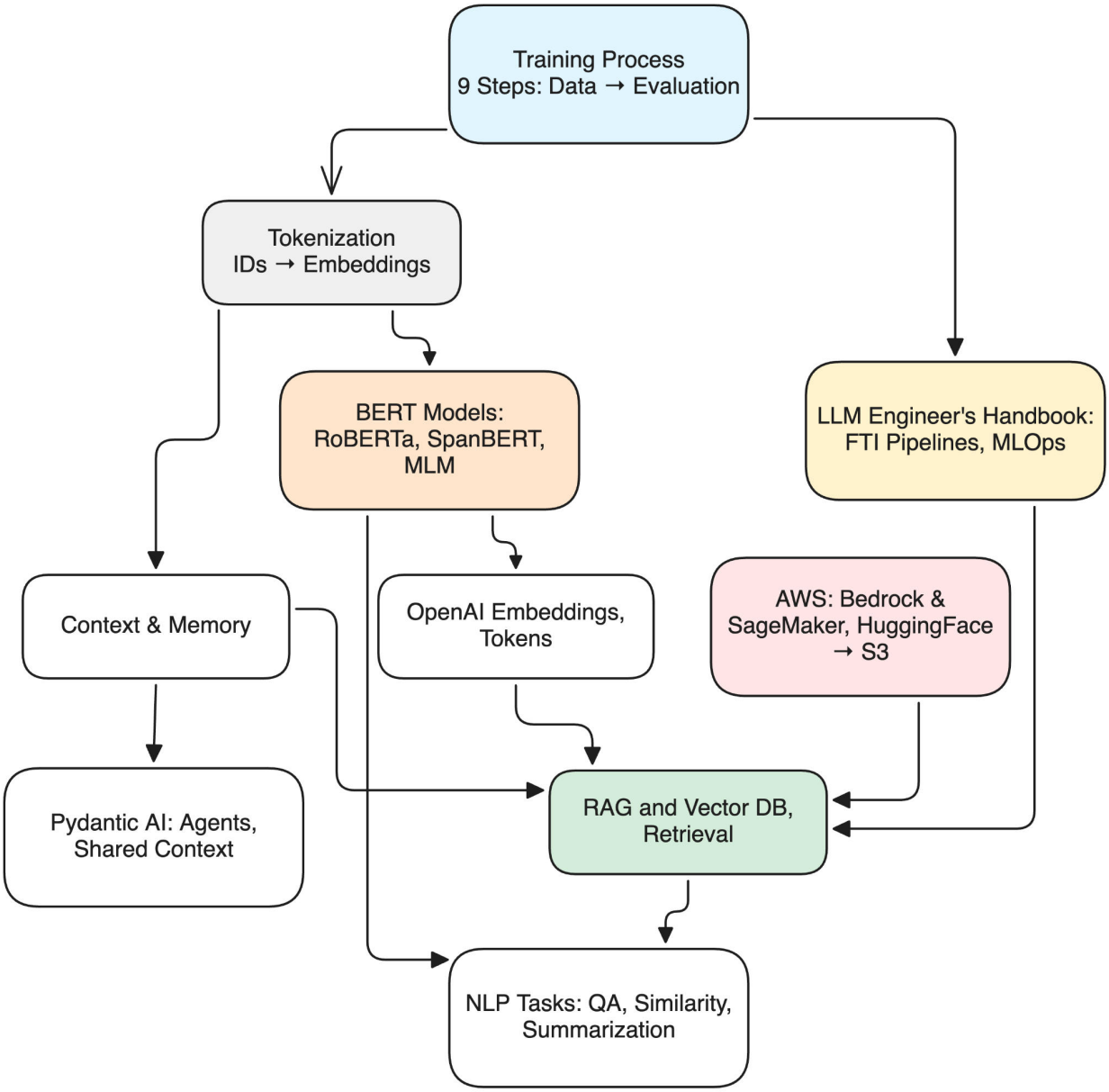
LLM and AI Notes

This is a personal collection of notes around LLM and AI in a notebook-style format.

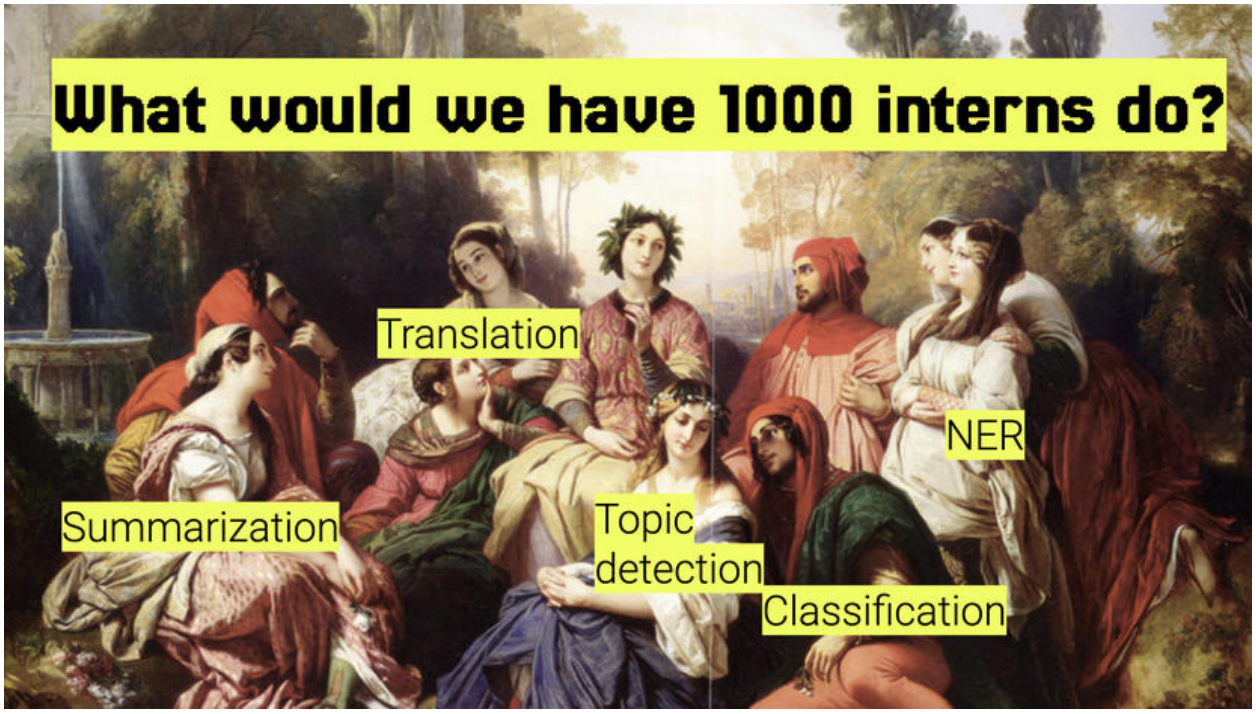
You can find the web version here.



Training Process of an AI Language Model

The goal of training an LLM is to maximize the likelihood of the correct token, which involves increasing its probability relative to other tokens. This way, we ensure the LLM consistently picks the target token (essentially the next word in the sentence) as the next token it generates.

Don't worry about LLMs | ★❤☆ Vicki Boykis ★❤☆



① Data Collection

Massive amounts of text data are collected from various sources like books, websites, articles, and other written content.

② Preprocessing

The collected data is preprocessed to clean and format it. This includes tasks like removing special characters, normalizing text, and splitting text into tokens (words or subwords).

③ Initialization

The model starts with random weights. These weights are parameters that the model will learn and adjust during training.

④ Forward Pass (Prediction)

In each iteration (or epoch), the model processes a batch of input text and makes predictions. For language models, this often involves predicting the next word in a sentence or filling in missing words.

⑤ Loss Calculation

The model's predictions are compared to the actual data to calculate a loss value, which measures how far off the predictions are from the true values. Common loss functions for language models include cross-entropy loss.

⑥ Backward Pass (Gradient Calculation)

The loss value is used to compute gradients, which indicate how the model's weights should be adjusted to reduce the error. This is done using a technique called backpropagation.

⑦ Parameter Update (Optimization)

The model's weights are updated using an optimization algorithm like Stochastic Gradient Descent (SGD) or Adam. These algorithms use the gradients to adjust the weights in a way that minimizes the loss.

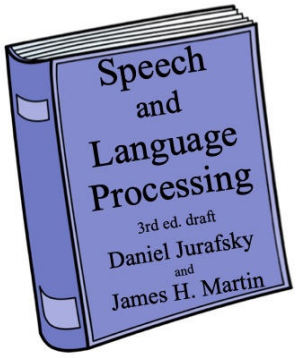
⑧ Iteration (Epochs)

This process is repeated for many iterations, with the model processing all the data multiple times (epochs). Over time, the model's weights are adjusted to better fit the training data, improving its predictions.

⑨ Evaluation

After training, the model is evaluated on separate validation and test datasets to assess its performance and ensure it generalizes well to new, unseen data.

Tokenization = the task of segmenting running text into words

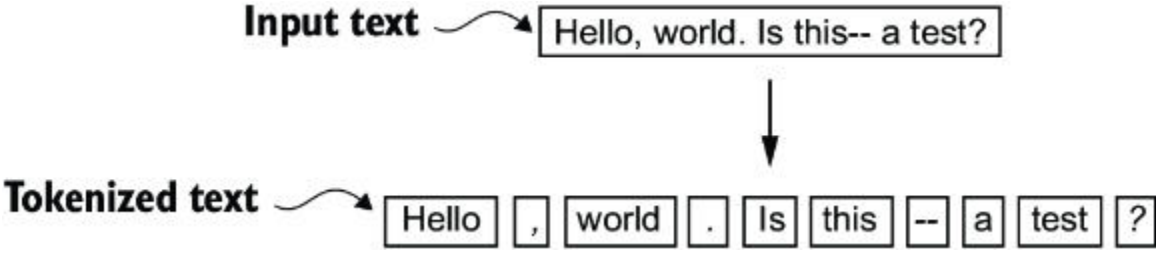


📑 Dan Jurafsky - Words and Tokens

Normalizing text means converting it to a more convenient, standard form. For example, most of what we are going to do with language relies on first separating out or tokenizing words or word parts tokenization from running text, the task of tokenization.

Another part of text normalization is lemmatization, the task of determining that two words have the same root, despite their surface differences. Lemmatization is essential for processing morphologically complex languages like stemming Arabic.

Stemming refers to a simpler version of lemmatization in which we mainly just strip suffixes from the end of the word. Text normalization also includes sentence segmentation: breaking up a text into individual sentences, using cues like sentence segmentation periods or exclamation points.



Converting tokens into token IDs

This conversion is an intermediate step before converting the token IDs into embedding vectors. An encode method splits text into tokens and carries out the string-to-integer mapping to produce token IDs via the vocabulary. A decode method

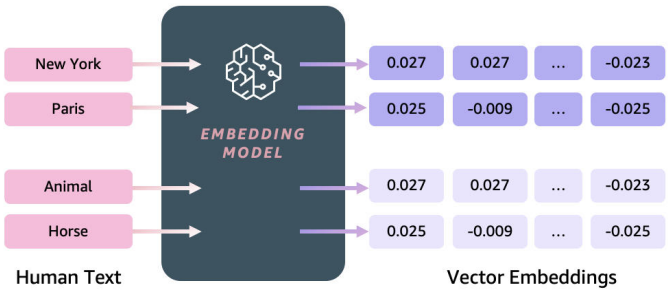
carries out the reverse integer-to-string mapping to convert the token IDs back into text.

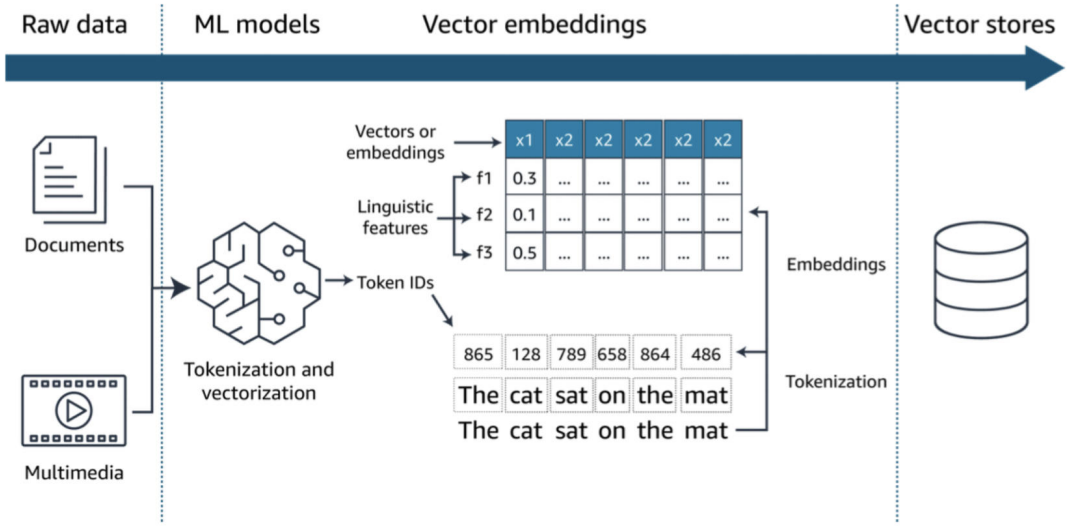
Creating token embeddings

The last step in preparing the input text for LLM training is to convert the token IDs into embedding vectors. The embedding layer is essentially a lookup operation that retrieves rows from the embedding layer's weight matrix via a token ID.

Embeddings

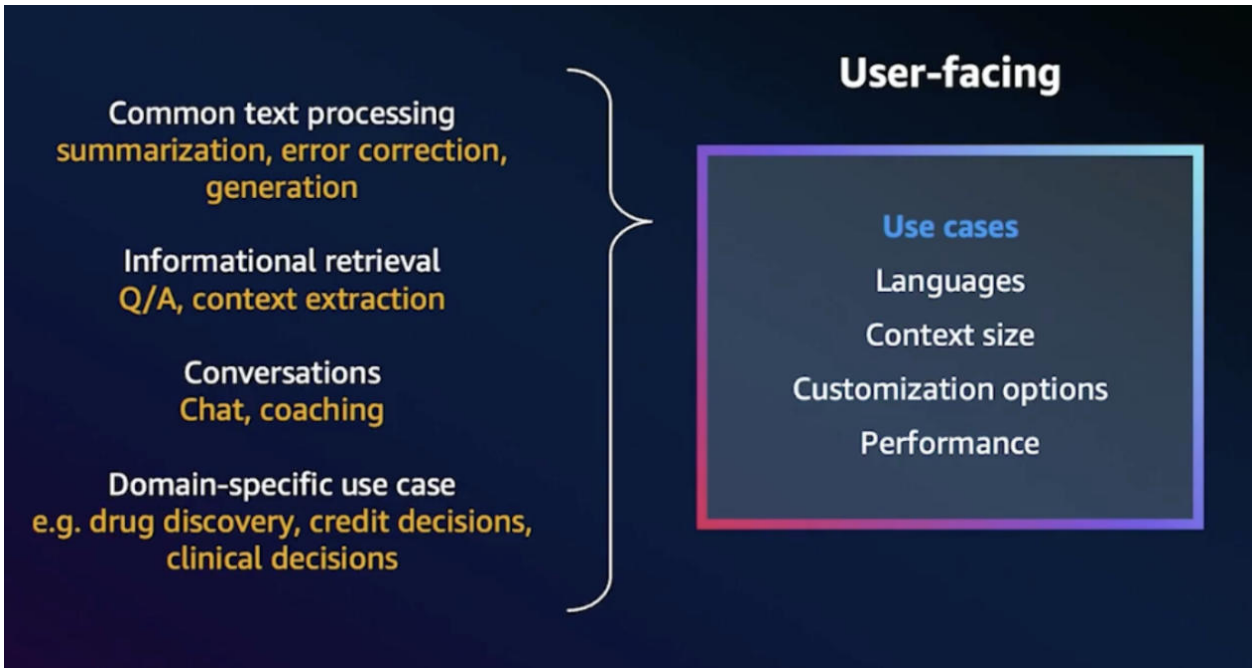
- Numerical representation of text (vectors) that captures semantics and relationships between words.
- Embedding models capture features and nuances of the text.
- Rich embeddings can be used to compare text similarity.
- Multilingual Text Embeddings can identify meaning in different languages.



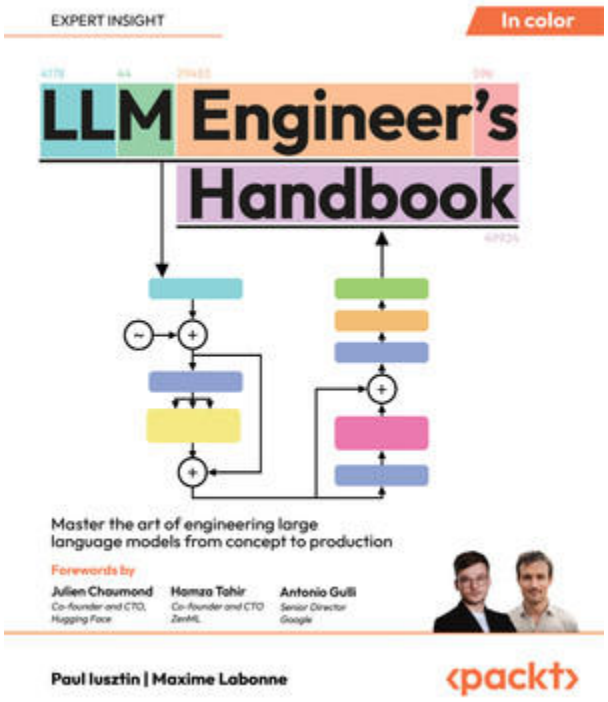


Tutorials

- GPT-2 Fine-Tuning w/ Hugging Face & PyTorch
- Training and Fine-Tuning Sentence Transformers Models
- Fine-tune a language model
- Developing an LLM: Building, Training, Finetuning



LLM Engineer's Handbook (Notes)



📑LLM Engineer's Handbook [Book]

The key to most successful ML products is to be **data-centric** and make your architecture **model-agnostic**.

The key of the LLM Twin stands in the following:

What data we **collect**	Data collection
How we **preprocess** the data	Data preprocessing
How we **feed** the data **into** the LLM	Data storage, versioning, and retrieval
How we **chain** prompts for the desired results	LLM fine-tuning and RAG
How we **evaluate** the generated content	Content generation evaluation

MLOps engineer's scope:

- Ingest, clean, and validate fresh data
- Training versus inference setups
- Compute and serve features in the right environment

- Serve the model in a cost-effective way
- Version, track, and share the datasets and models
- Monitor your infrastructure and models
- Deploy the model on a scalable infrastructure
- Automate the deployments and training

Model training is more of a research problem managed by a data science team (how to pick hyperparameters etc).

Feature-Training-Inference (FTI) pipelines

Most software applications can be split between a DB, business logic, and UI layer. In ML, feature, training, and inference pipelines (FTI pipelines). Each pipeline can be written using a different technology, by a different team, or scaled differently.

The data and feature pipelines will be **scaled horizontally** based on the CPU and RAM load. The training pipeline will be **scaled vertically** by adding more GPUs. The inference pipeline will be **scaled horizontally** based on the number of client requests.

Training ML models is an entirely **iterative** and **experimental process**. Unlike traditional software development, it involves running multiple parallel experiments, comparing them based on predefined metrics, and deciding which one should advance to production.

📑From MLOps to ML Systems with Feature/Training/Inference Pipelines - Hopsworks

RAG: The documents we add as context along the query represent our external state. Even in a fully automated ML system, it is recommended to have **a manual step** before accepting a new production model.

📑 Vector DB Comparison
📑 Rerankers and Two-Stage Retrieval | Pinecone
📑 Migrate to PyMongo Async

The context_length is a variable that represents the supported input size of the LLM. Here, we choose it similar to the maximum length of the input text.

Suppose you are working with a model that has the following specifications:

- Embedding dimensions: 768

- Context length: 2048 tokens

Memory Requirements:

- Each token is represented as a 768-dimensional vector.
- For 2048 tokens, the total memory required for the embeddings alone would be 2048×768 floating-point numbers.

Here are a few techniques to handle models with shorter windows:

- Chunking: Break text into smaller chunks and process them sequentially.
- Sliding Window: Move the model's "attention" across overlapping sections of the text.
- Retrieval-Augmented Generation (RAG): Retrieve relevant information dynamically without cramming everything into the context window.
- Fine-Tuning: Optimize models specifically for longer inputs.

Pydantic AI

Pydantic AI provides a type-safe framework for building AI agents with structured outputs and validation. Unlike raw API calls, Pydantic AI enforces data schemas and provides elegant abstractions for both synchronous and asynchronous interactions, making it easier to integrate LLMs into production applications with proper error handling and type checking.

Basic Usage

📑 Agents - Pydantic AI

This example demonstrates the basic patterns for running Pydantic AI agents with a locally hosted LLM via Ollama. The code illustrates three interaction modes: synchronous execution for simple queries, asynchronous execution for non-blocking operations, and streaming responses for real-time output generation.

```python
import os

from pydantic_ai import Agent
from pydantic_ai.models.openai import OpenAIModel
from pydantic_ai.providers.openai import OpenAIProvider

ollama_model = OpenAIModel(
    model_name="llama3.1:8b",
    provider=OpenAIProvider(base_url="http://localhost:11434/v1"),
    api_key=os.getenv("OPENAI_API_KEY", "ollama")
)
agent = Agent(ollama_model)

result_sync = agent.run_sync('What is the capital of Italy?')
print(result_sync.data)
#> Rome


async def main():
    result = await agent.run('What is the capital of France?')
    print(result.data)
    #> Paris

    async with agent.run_stream('What is the capital of the UK?') as response:
        print(await response.get_data())
        #> London

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```

Shared context

📑Messages and chat history - Pydantic AI

Maintaining conversation history is essential for building coherent multi-turn interactions where the agent needs to reference previous exchanges. This example shows how to pass message history between agent runs, enabling contextual conversations where follow-up questions naturally build on earlier responses without requiring the user to repeat information.

Python

```python
import os
import asyncio
from pydantic_ai import Agent
from pydantic_ai.models.openai import OpenAIModel
from pydantic_ai.providers.openai import OpenAIProvider

# Set up Ollama model via OpenAI-compatible endpoint
ollama_model = OpenAIModel(
    model_name="llama3.1:8b",
    provider=OpenAIProvider(base_url="http://localhost:11434/v1"),
    api_key=os.getenv("OPENAI_API_KEY", "ollama")
)

# Initialize the agent
wine_agent = Agent(
    ollama_model,
    system_prompt="You're a wine expert. Be conversational and helpful."
)

async def main():
    # 🍷 Step 1: Ask about Georgian wine
    print("🍷 Step 1: Ask about Georgian wine")
    result1 = await wine_agent.run("What is the most popular Georgian wine?")
    print("🤖 Agent:", result1.data)

    # 🍇 Step 2: Ask about red wine names – pass message history
    print("\n🍇 Step 2: Ask about red wine names")
    result2 = await wine_agent.run(
        "If it is red, what particular names would come to mind?",
        message_history=result1.new_messages()
    )
    print(f"🤖 Agent: {result2.data}")

    # 🍲 Step 3: Ask about food pairing – continue context
    print("\n🍲 Step 3: Ask about food pairing")
    result3 = await wine_agent.run(
        "What kind of food would go best with a wine of choice?",
        message_history=result2.new_messages()
    )
    print(f"🤖 Agent: {result3.data}")

if __name__ == "__main__":
    asyncio.run(main())
```

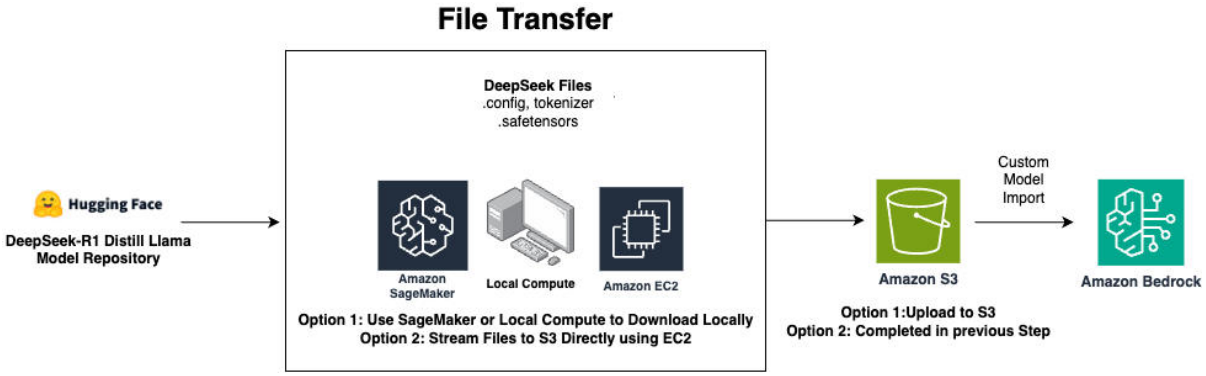
Amazon Bedrock and SageMaker AI

Amazon Bedrock

🤗 Loading Hugging Face Models to S3 for Bedrock

Amazon Bedrock supports custom model imports, enabling you to deploy open-source models from Hugging Face that aren't available in Bedrock's default catalog. This example demonstrates the complete workflow: authenticating with Hugging Face Hub, downloading model artifacts (in this case, the 70B parameter DeepSeek-R1 distilled model), uploading them to S3 for Bedrock access, and cleaning up local cache to reclaim storage space.

Deploy DeepSeek-R1 distilled Llama models with Amazon Bedrock Custom Model Import | Artificial Intelligence




```python
!pip install huggingface_hub

import getpass
import shutil

try:
```

```
    token = getpass.getpass()
except Exception as error:
    print('ERROR', error)
else:
    print('Password entered')

from huggingface_hub import login

# Log in programmatically
login(token=token)
print("✅ Successfully logged into Hugging Face!")

from huggingface_hub import snapshot_download

model_path =
snapshot_download(repo_id="deepseek-ai/DeepSeek-R1-Distill-Llama-70B")
print(f"Model downloaded to: {model_path}")

import boto3
import os

s3_client = boto3.client("s3")
bucket_name = "bedrock-models-111111111111"
s3_folder = "deepseek-70b/"

# Upload all model files to S3
for root, _, files in os.walk(model_path):
    for file in files:
        local_path = os.path.join(root, file)
        s3_path = os.path.join(s3_folder, os.path.relpath(local_path,
model_path))

        print(f"Uploading {local_path} to s3://{bucket_name}/{s3_path}...")
        s3_client.upload_file(local_path, bucket_name, s3_path)

print("✅ Model successfully uploaded to S3!")

# Path to Hugging Face cache folder
hf_cache_dir = "~/.cache/huggingface/hub"

# Delete cache folder completely
shutil.rmtree(hf_cache_dir, ignore_errors=True)
```

Using Local Models in Bedrock

Custom imported models in Bedrock are invoked using the same boto3 API as native models, requiring only the model's ARN for identification. This example shows how to send inference requests with configurable parameters like temperature and max tokens, handle the JSON response structure specific to imported models, and implement robust error handling with automatic retries.

Python

```python
import boto3
import json
from botocore.config import Config
from IPython.display import display, Markdown


def pretty_print_response(response):
    """
    Displays the LLM response in a clean, readable format using Markdown.
    """
    display(Markdown(f"**Response:**\n\n{response}"))

# AWS Region and Model ARN
REGION_NAME = "us-east-1"
MODEL_ID = "arn:aws:bedrock:us-east-1:111111111111:imported-model/rh1o7tgml7mr"
# 70B

# Retry configuration
config = Config(
    retries={"total_max_attempts": 10, "mode": "standard"}
)

# Create Bedrock runtime client
session = boto3.session.Session()
bedrock_client = session.client("bedrock-runtime", region_name=REGION_NAME,
config=config)

# Define the input prompt
prompt = (
    "Describe the significance of machine learning for risk management with 5
bullet points."
)

# Construct the request payload
request_payload = {
```

```
    "prompt": prompt,
    "temperature": 0.7
}

# Convert request to JSON
request_body = json.dumps(request_payload)
print("📨 Request Sent:\n", request_body)

try:
    # Invoke the Bedrock model
    response = bedrock_client.invoke_model(
        modelId=MODEL_ID,
        body=json.dumps({'prompt': prompt, 'max_gen_len': 2048}),
        accept="application/json",  # Ensure response is JSON
        contentType="application/json"  # Ensure request is JSON
    )

    # Decode the response
    response_body = json.loads(response["body"].read().decode("utf-8"))

    # Print the full response for debugging
    # print("📨 Full Response JSON:\n", json.dumps(response_body, indent=2))

    # Extract the correct key ("generation" instead of "generated_text")
    response_text = response_body.get("generation", "No 'generation' key found
in response")

    print("AI Response:\n", pretty_print_response(response_text))

except Exception as e:
    print(f"ERROR: Can't invoke '{MODEL_ID}'. Reason: {e}")
```

Amazon SageMaker

🤗 Serialize Hugging Face Model to S3 in SageMaker

Deploying fine-tuned models for production use requires packaging them in a format that inference endpoints can consume. This example demonstrates the complete workflow for serializing a Hugging Face sentiment classification model and storing it in S3, where it can be accessed by SageMaker endpoints, Lambda functions, or other AWS services. The process involves loading the fine-tuned model artifacts, bundling

both the model weights and tokenizer into a compressed archive following SageMaker's expected structure, and uploading to S3 for deployment.

```python
# Serialize to S3
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import boto3
import tarfile

def sentiment_analysis(text):
    inputs = tokenizer(text, return_tensors="pt")
    outputs = model(**inputs)
    logits = outputs.logits
    predicted_class = torch.argmax(logits, dim=1).item()

    # Map the predicted class to a sentiment label
    sentiment_labels = [1, 2, 3, 4, 5]
    sentiment = sentiment_labels[predicted_class]

    return sentiment

# Load the fine-tuned model and tokenizer
model_path = "tmp/artifacts/e5-small-fine-tuned/"
tokenizer = AutoTokenizer.from_pretrained('intfloat/e5-small-v2')
model = AutoModelForSequenceClassification.from_pretrained(model_path)

# Package our model and tokenizer for Gradio
local_model_path = "tmp/sentiment-analysis-model"
model.save_pretrained(local_model_path)
tokenizer.save_pretrained(local_model_path)

# Save as a tar.gz object
model_artifact_path = "tmp/sentiment-analysis-model.tar.gz"
with tarfile.open(model_artifact_path, "w:gz") as tar:
    tar.add(local_model_path, arcname="sentiment-analysis-model")

# Define S3 bucket location
s3 = boto3.client('s3')
bucket_name = "my-s3-bucket"
artifact_folder = "sentiment-analysis/artifacts/e5-small-fine-tuned"
# Upload the model artifact to S3
s3.upload_file(model_artifact_path, bucket_name,
f"{artifact_folder}/sentiment-analysis-model.tar.gz")
```

🤗 Hugging Face - Load Model from S3

After packaging models to S3, they can be retrieved and loaded for inference in any environment. This example demonstrates downloading the compressed model artifact, extracting the archive, and loading both model and tokenizer into memory for prediction.

```python
# Load model and tokenizer from S3
import tarfile
import boto3
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# Set up the S3 client
s3 = boto3.client('s3')

# Define the S3 bucket and artifact location
bucket_name = "my-s3-bucket"
artifact_folder = "sentiment-analysis/artifacts/e5-small-fine-tuned"
model_artifact_name = "sentiment-analysis-model.tar.gz"

# Download the model artifact from S3
local_model_artifact_path = "tmp/sentiment-analysis-model.tar.gz"
s3.download_file(bucket_name, f"{artifact_folder}/{model_artifact_name}",
local_model_artifact_path)

# Extract the model and tokenizer files
with tarfile.open(local_model_artifact_path, "r:gz") as tar:
    tar.extractall("tmp/")

# Load the model and tokenizer
model =
AutoModelForSequenceClassification.from_pretrained("tmp/sentiment-analysis-mode
l")
tokenizer = AutoTokenizer.from_pretrained("tmp/sentiment-analysis-model")
```

Deploying as a Gradio App

Gradio provides a simple way to create web interfaces for model inference. This section presents two deployment patterns: a static version with explicit submission for controlled predictions, and a real-time version with live updates as users type. Both include basic authentication and can run locally or be deployed to cloud platforms.

Version 1 (static)

The static version uses Gradio's Blocks API to create a custom layout with explicit user control. Users enter text into the input field and click the "Analyze Sentiment" button to trigger prediction.

```python
import torch
import gradio as gr

def sentiment_analysis(text):
    inputs = tokenizer(text, return_tensors="pt")
    outputs = model(**inputs)
    logits = outputs.logits
    predicted_class = torch.argmax(logits, dim=1).item()

    # Map the predicted class to a sentiment label
    sentiment_labels = [1, 2, 3, 4, 5]
    sentiment = sentiment_labels[predicted_class]

    return sentiment

with gr.Blocks() as demo:
    gr.Markdown("# Sentiment Analysis Demo")

    with gr.Row():
        text_input = gr.Textbox(label="Enter Text")
        submit_button = gr.Button("Analyze Sentiment")

    sentiment_output = gr.Textbox(label="Sentiment")

    submit_button.click(sentiment_analysis, inputs=text_input,
outputs=sentiment_output)

demo.launch(auth=("admin", "admin"))
```

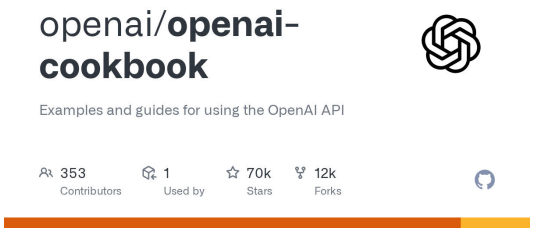
Version 2 (real time)

The real-time version uses Gradio's Interface API with live=True to provide instant feedback as users type. This creates a more interactive experience where sentiment predictions update dynamically.

Python

```python
import pandas as pd
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
import gradio as gr
import numpy as np

# Function to get the prediction for the current input text
def predict_text(text, tokenizer, model):
    inputs = tokenizer(text, return_tensors="pt", padding=True,
truncation=True, max_length=512)
    with torch.no_grad():
        outputs = model(**inputs)
    logits = outputs.logits
    predicted_class = logits.argmax(dim=-1).item()
    return predicted_class + 1  # Adjusting class index to 1-based scale

# Function to get sentiment prediction and scores for Gradio
def get_sentiment_prediction(text):
    return "" if not text.strip() else predict_text(text, tokenizer, model)

# Gradio interface
def predict_and_display(text):
    final_score = get_sentiment_prediction(text)
    return f"Text Sentiment Score: {final_score}"

demo_rt = gr.Interface(
    fn=predict_and_display,
    inputs=gr.Textbox(lines=2, placeholder="Enter text here...", label="Text
Input"),
    outputs=gr.Textbox(lines=2, label="Sentiment Score"),
    live=True,
    title="Real-time Sentiment Analysis",
    description="Enter text and see real-time sentiment predictions for the
entire text.",
)

demo_rt.launch(auth=("admin", "admin"))
```

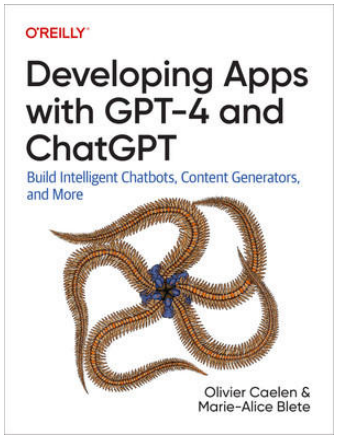
OpenAI



OpenAI cookbook extends beyond basic API usage to cover critical implementation details such as using embeddings for classification, chunking strategies, and more.

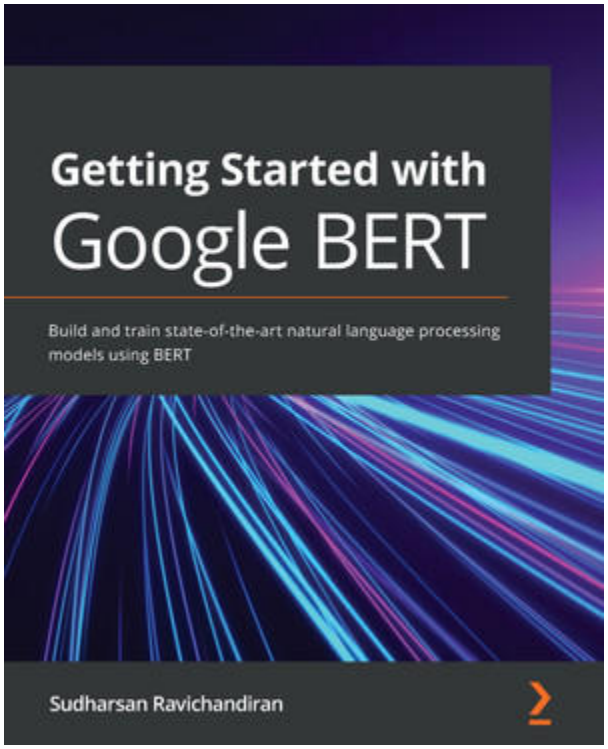
> Embedding texts that are longer than the model's maximum context length
>
> Embeddings - OpenAI API
>
> What are tokens and how to count them?
>
> How to count tokens with Tiktoken
>
> Are spaces and new lines counts as tokens
>
> Tokenizer - OpenAI API

This book nicely complements OpenAI materials:



📑Developing Apps with GPT-4 and ChatGPT [Book]

BERT



📑 Getting Started with Google BERT [Book]

RoBERTa is a variant of BERT and it uses only the masked language modeling task for training. Unlike BERT, it uses dynamic masking instead of static masking and it is trained with large batch sizes. It uses the byte-level BPE as a tokenizer and it has a vocabulary of size 50K.

SpanBERT (QA)

Question answering tasks require models to extract specific text spans from a given context. SpanBERT is specifically designed for span selection tasks, making it ideal for extracting answers from documents.

```python
from transformers import pipeline

qa_pipeline = pipeline(
    "question-answering",
    model="mrm8488/spanbert-large-finetuned-squadv2",
```

```
    tokenizer="SpanBERT/spanbert-large-cased"
)

results = qa_pipeline({
    'question': "what is machine learning?",
    'context': "Machine learning is a subset of artificial intelligence. It is
widely for creating a variety of applications such as email filtering and
computer vision"
})

# cls_rep = representation[:, 0, :]  # CLS token
```

Masked Language Modeling (MLM)

Masked language modeling is the core pre-training task for BERT-style models, where the model learns to predict masked tokens based on surrounding context. This capability can be used to fill in missing words or validate text completeness.

Python

```python
from transformers import pipeline

predict_mask = pipeline(
    "fill-mask",
    model= "dccuchile/bert-base-spanish-wwm-uncased",
    tokenizer="dccuchile/bert-base-spanish-wwm-uncased"
)
# sentence "todos los caminos llevan a roma"
result = predict_mask('[MASK] los caminos llevan a Roma')
print(result)
# {'sequence': '[CLS] todos los caminos llevan a roma [SEP]', 'score':
0.9719983339309692, 'token': 1399, 'token_str': 'todos'}
```

Cosine Similarity

Sentence embeddings allow us to measure semantic similarity between texts by representing them as dense vectors. Computing the cosine similarity between these embeddings quantifies how closely related two sentences are in meaning.

```python
model = SentenceTransformer('bert-base-nli-mean-tokens')
sentence1 = 'It was a great day'
sentence2 = 'Yesterday was awesome'
sentence1_representation = model.encode(sentence1)
sentence2_representation = model.encode(sentence2)
cosine_sim =
util.pytorch_cos_sim(sentence1_representation,sentence2_representation)
print(cosine_sim)
```

Similar Sentence Search

In many applications like customer support or FAQ systems, we need to match user queries to the most relevant pre-existing questions. By encoding both the input question and a database of reference questions as embeddings, we can efficiently find the closest semantic match.

```python
from sentence_transformers import SentenceTransformer, util
import numpy as np

model = SentenceTransformer("bert-base-nli-mean-tokens")

master_dict = [
    "How to cancel my order?",
    "Please let me know about the cancellation policy?",
    "Do you provide refund?",
    "what is the estimated delivery date of the product?",
    "why my order is missing?",
    "how do i report the delivery of the incorrect items?",
]
inp_question = "When is my product getting delivered?"
# Embed representations
inp_question_representation = model.encode(inp_question,
convert_to_tensor=True)
master_dict_representation = model.encode(master_dict, convert_to_tensor=True)
# Compute similarity
similarity = util.pytorch_cos_sim(
    inp_question_representation, master_dict_representation
)
# Get the result
print(
```

```
    "The most similar question in the master dictionary to given input question
is:",
    master_dict[np.argmax(similarity.cpu())],
)
```

Text summarization

Generating concise summaries from longer documents is a common need in information processing. BART combines bidirectional encoding with autoregressive decoding, making it particularly effective for abstractive summarization tasks.

bart-large-cnn is the pre-trained BART large model for the text summarization.

Python

```python
from transformers import BartTokenizer, BartForConditionalGeneration

model = BartForConditionalGeneration.from_pretrained('facebook/bart-large-cnn')
tokenizer = BartTokenizer.from_pretrained('facebook/bart-large-cnn')

text = """Machine learning (ML) is the study of computer algorithms that
improve automatically through experience.It is seen as a subset of artificial
intelligence. Machine learning algorithms build a mathematical model based on
sample data, known as training data, in order to make predictions or decisions
without being explicitly programmed to do so.Machine learning algorithms are
used in a wide variety of applications, such as email filtering and computer
vision, where it is difficult or infeasible to develop conventional algorithms
to perform the needed tasks."""

inputs = tokenizer([text], max_length=1024, return_tensors='pt')
summary_ids = model.generate(inputs['input_ids'], num_beams=4, max_length=100,
early_stopping=True)
summary = ([tokenizer.decode(i, skip_special_tokens=True,
clean_up_tokenization_spaces=False) for i in summary_ids])
```

Retrieval-Augmented Generation (RAG)

Using DuckDB + Ibis for RAG

SQLiteVSS — 🦜🔗 LangChain documentation

Streamline your LangChain deployments with LangServe - DEV Community

Sentence Transformers on Hugging Face | 🦜🔗 LangChain

MongoDB RAG notebooks

Chroma

PostgreSQL Extensions: Turning PostgreSQL Into a Vector Database With pgvector

Pandas AI with Bedrock

Analyzing tabular data typically requires programming expertise, but large language models can translate natural language questions into executable pandas operations. This example demonstrates how to combine PandasAI with AWS Bedrock to create an intelligent dataframe wrapper that responds to conversational queries.

📑 Examples - PandasAI

Python

```python
from pandasai import SmartDataframe
from pandasai.llm import BedrockClaude
import boto3

session = boto3.Session(profile_name="default")
bedrock_runtime_client = session.client(
    "bedrock-runtime",
)

llm = BedrockClaude(bedrock_runtime_client)
df = SmartDataframe("../csv/my-excel-file.xlsx", config={"llm": llm})
```